

CVM++

Setup, structure, glossary, and construction path

From clone to running cvmpp — with every term explained

Introduction

This document explains **what CVM++ is**, **how to build it**, **how the repository is organized**, the **order used to implement each module**, and a **glossary** of every technical term in the project.

Note

Read the **Architecture and Workflow** PDF for byte-level execution traces, opcode tables, and VM internals.

What is CVM++?

CVM++ is a **scripting language** and **toolchain** written in **C++17**. You author `.cvm` text files; the `cvmp` program compiles and runs them without LLVM, Python, or an external interpreter library.

Execution pipeline



Component	Role
Language	.cvm scripts: variables, functions, control flow, I/O
Lexer	Splits source into tokens
Parser	Builds an AST (syntax tree)
Compiler	Emits stack bytecode in <code>std::vector<uint8_t></code>
VM	Executes bytecode on an operand stack
CLI / REPL	Terminal interface: files, debug, interactive mode

Prerequisites

Tool	Purpose
C++17 compiler	GCC 9+, Clang 10+, or MSVC 2019+
Make	<code>make</code> , <code>make verify</code>
CMake 3.16+	Optional build path
Git	Clone and publish the repository

Quick tip

Regenerate PDFs after editing: `typst compile docs/typst/01-project-build.typ docs/pdf/...` or run `./scripts/build-docs.sh`.

Quick start

```
git clone https://github.com/Dragon9-9/cvmpp.git
cd cvmpp
make
./build/cvmpp examples/hello.cvm
./build/cvmpp -d examples/functions.cvm
./build/cvmpp
make verify
```

Flag	Effect
-d	Print tokens, AST, and bytecode
-q	Quiet: program output only
-c	Compile only; skip VM

Exit codes: 0 success · 1 compile error · 2 runtime error

Repository structure

```
cvmpp/
├─ include/cvm++/      Public headers
├─ src/                Implementations (.cpp)
├─ examples/          Sample .cvm programs
├─ docs/pdf/          PDF guides (this document)
├─ docs/typst/        Typst sources for PDFs
├─ scripts/           verify.sh, build-docs.sh
├─ Makefile
├─ CMakeLists.txt
└─ .github/workflows/ CI (build + verify)
```

Module reference

Files	Module	Responsibility
lexer.cpp	Lexer	Characters → tokens
parser.cpp	Parser	Tokens → AST
compiler.cpp	Compiler	AST → bytecode
vm.cpp	VM	Bytecode → execution
compile.cpp	Frontend	Wires lexer → parser → compiler
main.cpp	CLI	REPL, file runner, flags
ui.cpp	UI	Colored diagnostics and debug tables

How the project was built

Follow this order when extending or reimplementing the system.

Phase 1 — Foundation

source_loc (line/column), diagnostic (phased errors), token, lexer, ui, minimal main that tokenizes input.

Phase 2 — Parser and AST

ast.hpp (expression and statement nodes), recursive-descent parser, ast_print for debug output.

Phase 3 — Bytecode compiler

opcode enum, BytecodeChunk + writer, jump patching for if/while, name interning pool.

Phase 4 — Virtual machine

VmValue as std::variant<int64_t, bool>, stack interpreter loop, globals, runtime failsafes.

Phase 5 — Product features

REPL, file mode, functions (fn, return, CALL/RETURN), examples, make verify, GitHub Actions CI.

Note

Function bodies are emitted **before** main bytecode. A JUMP at offset 0 skips them so execution starts at the top-level script — not inside a function body.

Language reference

```
let x = 10;
fn add(a, b) { return a + b; }
if (x < 100) { print add(x, 1); }
while (x > 0) { x = x - 1; }
let n = input;
print n;
```

Feature	Syntax
Integers / booleans	42, true, false
Variables	let x = ...; and x = ...;
Functions	fn f(a) { ... } and f(1, 2)
Control flow	if / else, while
Operators	+ - * / == != < > <= >=
I/O	print expr;, let x = input;
Comments	// line comment

Not included: strings, floats, arrays, for loops, modules.

Example programs

Script	Demonstrates
hello.cvm	Basics: let, if, print
functions.cvm	Recursive fn / return
factorial.cvm	while loop
comparisons.cvm	!=, <=, >=
div_by_zero.cvm	Runtime error, exit code 2

Glossary

Term	Definition
Token	Classified source unit: keyword, literal, operator, or punctuation
AST	Abstract Syntax Tree — program structure after parsing
Bytecode	Compact uint8_t instruction stream for the VM
Opcode	Instruction tag: e.g. Add, Call, JumpIfFalse
Stack	LIFO structure holding intermediate expression values
Call frame	Return address + local variable slots during a call
IP	Instruction pointer — index into the bytecode array
Name pool	chunk.names — variable names referenced by index
Diagnostic	Error with phase, location, message, and hint
VmSession	REPL-only map persisting globals between lines
Front end	Lexer + parser + compiler in one pipeline

Troubleshooting

Problem	Solution
cvmpp not found	Run make; invoke ./build/cvmpp
make verify fails	Run ./build/cvmpp -d examples/<file>.cvm
REPL state lost	File runs reset VM; REPL keeps session globals